

LEVEL 18

Chapter 21: Real-Time System Design

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 21: Real-Time System Design

← [Chapter 20: FinTech Deep Dive](#) · [Index](#) · Next → [Chapter 22: Search Systems](#)

The mechanics here already surfaced piecemeal in the WhatsApp, Uber, and chat-at-scale problems ([Chapter 17, Problem 9](#); [Chapter 18, Problems 17, 29](#)) — this chapter is the consolidated reference for the transport choice and scaling techniques underneath all of them.

21.1 Choosing a Real-Time Transport

	Long Polling	SSE (Server-Sent Events)	WebSocket
Direction	Client → Server request, server holds it open until there's data, then client immediately re-requests	Server → Client only, one-way stream over HTTP	Full-duplex, both directions, persistent connection
Protocol	Plain HTTP, repeated requests	Plain HTTP, single long-lived connection	Its own protocol, upgraded from HTTP (Chapter 1.3)
Overhead	Higher — repeated HTTP request/response cycle, headers each time	Low — one connection, server just keeps writing	Lowest per-message, but holds a persistent connection per client
Browser/infra support	Universal, works through almost any proxy/firewall	Very good, simpler than WebSocket to reason about, auto-reconnect built into the browser API	Universal in modern browsers, but some older corporate proxies/firewalls historically blocked it
Best for	Simple, infrequent updates where WebSocket infrastructure isn't justified	Server-push-only use cases: live dashboards, live scores, notification streams — where the client never needs to send data back over the same channel	Bidirectional, frequent, low-latency needs: chat, live location, multiplayer/collaborative features

The decision rule to say out loud: "Does the client need to *send* data over this same real-time channel, not just receive it?" If yes (chat, live cursor position in collaborative editing) → WebSocket. If it's purely server-to-client (a live dashboard, a live notification feed, a stock ticker) → **SSE is usually the simpler, sufficient choice** — a genuinely underused option that many candidates skip straight past in favor of WebSocket even

when it's not needed, which is worth explicitly avoiding (reaching for the more complex tool without justifying it is the same anti-pattern as reaching for Kafka unprompted, Chapter 14.4).

Interview question: "You're building a live sports score dashboard. WebSocket or SSE?"

Ideal senior answer: "SSE — the client only ever receives updates, it never needs to send anything back over this channel. SSE gives me that with plain HTTP, automatic reconnection handling built into the browser's `EventSource` API, and less infrastructure complexity than running a WebSocket connection pool for a purely one-directional need. I'd reach for WebSocket only if this evolved to need bidirectional interaction — say, users submitting live predictions that need to be acknowledged in real time."

21.2 Connection Infrastructure at Scale — The Pattern Recap

Every real-time system at scale converges on the same shape you saw in [Chapter 17's WhatsApp problem](#): a pool of **connection/gateway servers** holding persistent connections, a **presence/routing layer** (commonly Redis, mapping `user_id` → `which gateway instance they're connected to`) so a message from anywhere in the system can find the right live connection, and a **message broker** decoupling senders from the specific gateway a recipient happens to be attached to. This is worth stating as a reusable, named pattern rather than re-deriving it from scratch every time a real-time problem appears — recognizing and naming pattern reuse is itself a strong signal (Chapter 18's framing throughout).

The one number worth knowing: a single connection server can typically hold anywhere from tens of thousands to low hundreds of thousands of concurrent WebSocket connections depending on message frequency and payload size (memory per connection, not CPU, is usually the binding constraint) — so at real scale (millions of concurrent users), you're always talking about a *pool* of connection servers, never one, and the presence/routing layer is not optional infrastructure, it's the load-bearing piece that makes horizontal scaling of stateful connections possible at all.

21.3 Presence

The problem: showing "who's online" accurately, cheaply, at scale — already touched on in [Chapter 18, Problem 29](#)'s Discord-scale discussion; here's the general pattern independent of chat specifically.

- **Naive approach (don't do this at scale):** broadcast every user's online/offline state change to every other user who might care, immediately. This is a fan-out problem (Chapter 13.3) that scales terribly — a popular user's presence changing could trigger millions of tiny broadcast messages.
 - **Better approach:** presence is stored centrally (a `user_id → {status, last_seen}` record, commonly in Redis with a TTL — a connection server periodically "heartbeats" on behalf of connected users, and absence of a heartbeat within the TTL window is treated as "gone offline," which elegantly handles ungraceful disconnects like a phone losing signal, not just clean disconnects). Other clients **pull** presence on demand (opening a contact list) rather than having it pushed to them continuously, and only clients *actively viewing* a relevant view (a specific open chat, a specific channel's member list) receive live push updates for that narrow scope — bounding the fan-out to only where it's actually needed, right now, rather than broadcasting globally.
-

21.4 Real-Time Location Tracking

Already covered in depth via [Chapter 18, Problems 17 and 23](#) (Uber's geospatial matching) — the consolidated technique list: **UDP or a lightweight transport over WebSocket** for the ping itself (Chapter 1.1's loss-tolerance reasoning), a **high-throughput ingestion pipeline** (Kafka-class) absorbing the sustained ping volume, a **geospatial index** (geohash/S2) held largely in-memory for fast nearest-neighbor queries, and **only the latest position matters** for live tracking — this is explicitly *not* a domain that needs durable long-term storage of every historical ping at full fidelity (though a downsampled/aggregated history might be retained separately for analytics or trip-replay features, which is a different, lower-priority data path).

Streaming a tracked location to a specific viewer (a customer watching their delivery partner's live position) reuses the exact connection-server-plus-routing pattern from Section 21.2 — the delivery partner's location service publishes position updates, which get routed specifically to the one customer connection currently watching that one specific delivery, a narrow, targeted push rather than any kind of broadcast.

21.5 Live Dashboards

A somewhat different real-time shape worth distinguishing explicitly: a live dashboard (an ops/metrics dashboard, a live sales counter) typically has **many viewers watching the same aggregated data**, rather than each user needing their own personalized stream — this changes the efficient design meaningfully.

The efficient pattern: rather than each connected viewer triggering its own independent computation of the current aggregate, compute the aggregate **once**, on a schedule or as new underlying events arrive (e.g., a streaming aggregation job, similar in shape to Chapter 18's surge-pricing zone aggregation), and **fan out the single computed result to all currently-watching connections** — this is a genuine one-to-many broadcast, unlike the location-tracking case above, and SSE (Section 21.1) is very often the right transport specifically because dashboards are almost always receive-only from the client's perspective.

Interview question: "1,000 ops engineers have your internal metrics dashboard open simultaneously. How do you avoid computing the same aggregate 1,000 times?"

Ideal senior answer: "Compute the aggregate once, centrally — a streaming job or a scheduled recomputation, however fresh the requirement demands — and push that single result to all connected dashboard clients via a pub/sub fan-out (SSE or WebSocket broadcast from a small set of gateway connections holding all 1,000 viewers). Each client re-computing or re-querying independently would multiply load by 1,000 for zero additional value, since they're all looking at the exact same number."

21.6 Scaling Checklist for Any Real-Time Feature

When a real-time requirement shows up inside a larger design problem, work through this checklist out loud:

1. **Transport:** bidirectional (WebSocket) or server-push-only (SSE)? (Section 21.1)
2. **Fan-out shape:** one-to-one (a specific delivery tracked by one customer), one-to-few (a group chat), or one-to-many (a live dashboard, a broadcast channel)? — this single answer determines almost everything else about the design.
3. **Connection routing:** do you need a presence/routing layer to find "which server is this specific recipient connected to"? (Almost always yes, past a single-server scale.)
4. **Durability requirement:** does a message need to survive the recipient being offline (WhatsApp-style — yes, persist and redeliver) or is it fine to simply miss it if you weren't connected at that instant (a live cursor position, a location ping — no, latest-value-wins is fine)?

5. **Ordering requirement:** does this data need strict per-entity ordering (chat messages in one conversation) or is out-of-order/latest-wins acceptable (location pings)?

Answering these five questions, explicitly, in order, is a compact, repeatable way to derive the right real-time architecture for almost any unfamiliar problem an interviewer hands you — which is exactly the "derive, don't memorize" philosophy this whole roadmap is built around.

Next → [Chapter 22: Search Systems](#) — *Elasticsearch internals, inverted indexes, ranking, autocomplete, and designing search for e-commerce/job platforms.*